Concurrency control

Concurrency control

- Concurrency controlling techniques ensure that multiple transactions are executed simultaneously while maintaining the ACID properties of the transactions.

Concurrency control

- Simultaneous execution of transactions over a shared database can create several data integrity and consistency problems.
  - Lost Update
  - Dirty Read Problems
  - Incorrect Summary Problems
  - Unrepeatable Read Problem

THE LOST UPDATE PROBLEM

Transaction T:		Transaction U:	
balance = b.getBalance();		*balance = b.getBalance();*	
*b.setBalance(balance*1.1);*		*b.setBalance(balance*1.1);*	
a.withdraw(balance/10)		*c.withdraw(balance/10)*	
balance = b.getBalance();	$200		
		balance = b.getBalance();	$200
		*b.setBalance(balance*1.1);*	$220
*b.setBalance(balance*1.1);*	$220		
a.withdraw(balance/10)	$80		
		c.withdraw(balance/10)	$280

Figure 13.5

- the initial balances of accounts A, B, C are $100, $200. $300
- both transfer transactions increase B's balance by 10%

- the net effect should be to increase *B* by 10%
- twice - 200, 220, 242.
- but it only gets to 220. *U*'s update is lost.

A SERIALLY EQUIVALENT INTERLEAVING OF *T* AND *U* (LOST UPDATES CURED)

Transaction *T*:		**Transaction *U*:**	
balance = b.getBalance()		*balance = b.getBalance()*	
*b.setBalance(balance*1.1)*		*b.setBalance(balance*1.1)*	
a.withdraw(balance/10)		*c.withdraw(balance/10)*	
balance = b.getBalance()	$200		
*b.setBalance(balance*1.1)*	$220		
		balance = b.getBalance()	$220
		*b.setBalance(balance*1.1)*	$242
a.withdraw(balance/10)	$80		
		c.withdraw(balance/10)	$278

Figure 13.7

- if one of *T* and *U* runs before the other, they can't get a lost update,
- the same is true if they are run in a serially equivalent ordering

- Their access to *B* is serial, the other part can overlap

THE INCONSISTENT RETRIEVALS PROBLEM

- Inconsistent retrievals occur when a retrieval transaction observes values that are involved in an ongoing updating transaction.
- Consider a situation, where one transaction is applying the aggregate function on some records while another transaction is updating these records.
- The aggregate function may calculate some values before the values have been updated and others after they are updated.

THE INCONSISTENT RETRIEVALS PROBLEM

Transaction *V*: *a.withdraw(100)* *b.deposit(100)*		Transaction *W*: *aBranch.branchTotal()*	
a.withdraw(100);	$100		
		total = a.getBalance()	$100
		total = total+b.getBalance()	$300
		total = total+c.getBalance()	
b.deposit(100)	$300	:	

Figure 13.6

- the initial balances of accounts A and B are both $200
- *V* transfers $100 from *A* to *B* while *W* calculates branch total (which should be $400)

- we see an inconsistent retrieval because *V* has only done the withdraw part when *W* sums balances of *A* and *B*

A SERIALLY EQUIVALENT INTERLEAVING OF *V* AND *W* (INCONSISTENT RETRIEVALS CURED)

Transaction *V*: *a.withdraw(100);* *b.deposit(100)*		**Transaction *W*:** *aBranch.branchTotal()*	
a.withdraw(100);	$100		
b.deposit(100)	$300		
		total = a.getBalance()	$100
		total = total+b.getBalance()	$400
		total = total+c.getBalance()	
		...	

Figure 13.8

- if *W* is run before or after *V*, the problem will not occur
- therefore it will not occur in a serially equivalent ordering of *V* and *W*
- the illustration is serial, but it need not be

- we could overlap the 1st line of *W* with the 2nd line of *V*